



Cycle: Ingénieurs (GI), Niveau: 1^{ème} année (GI1)

Module : Structure de données et Programmation avancée en C

Les fichiers

Professeur : Ahmad EL ALLAOUI

Département d'Informatique, FST Errachidia

a.elallaoui@umi.ac.ma

Année universitaire : 2025/2026

Les fichiers

2

- ▶ Les données stockées en mémoire centrale sont les variables des différents programmes et ne durent que le temps de l'exécution d'un programme.
- ▶ Les tableaux stockent les données en mémoire centrale. Pour mémoriser des données de manière permanente, il faut les stocker sur un disque (ou un périphérique tel qu'une clef USB).
- ▶ Un fichier est une série de données stockées sur un disque ou dans un périphérique de stockage. Un fichier texte est un fichier qui contient du texte ASCII.
- ▶ lecture dans un fichier : transfert de données du fichier vers la RAM.
- ▶ écriture dans un fichier : transfert de données de la RAM vers le fichier.

Trois flots standard peuvent être utilisés en C sans qu'il soit nécessaire de les ouvrir ou de les fermer :

- `stdin` (standard input) : unité d'entrée (par défaut, le clavier) ;
- `stdout` (standard output) : unité de sortie (par défaut, l'écran) ;
- `stderr` (standard error) : unité d'affichage des messages d'erreur (par défaut, l'écran).

Différence entre fichiers texte

- ▶ **Un fichier binaire** contient du **code binaire**. On ne peut pas visualiser son contenu avec un éditeur de texte. Lorsqu'une variable est écrite dans un fichier binaire, on écrit directement la valeur exacte de la variable, telle qu'elle est **codée en binaire en mémoire**. Cette manière de stocker les données est plus précise et plus compacte pour coder des nombres. Les fonctions de lecture et d'écriture dans un fichier binaire sont **fread** et **fwrite** qui lisent et écrivent des blocs de données sous forme binaire.

- ▶ Non lisible par le programmeur
- ▶ Uniquement lecture/écriture par un programme

- ▶ Un fichier texte contient du **texte ASCII**. Lorsqu'un fichier texte contient des nombres, ces nombres sont codés sous forme de texte à l'aide des caractères '1', '2', etc. Dans ce format, chaque chiffre prend 1 octet en mémoire. On peut visualiser le contenu d'un fichier texte avec un éditeur de texte tel que, **bloccnote**. Les fonctions de lecture et écriture dans un fichier texte (**fscanf**, **fprintf**...) sont analogues aux fonctions de lecture et d'écriture de texte dans une console scanf et printf.

- ▶ lisible par le programmeur
- ▶ lecture/écriture par un programme et par éditeur

Les fichiers

4

Pour manipuler un fichier, on a besoin d'un certain nombre d'informations : **l'adresse de l'endroit** de la RAM où se trouve le fichier, la position de la tête de lecture, le mode d'accès au fichier (lecture ou écriture) ...

Ces informations sont rassemblées dans une structure dont le type, **FILE ***, est défini dans stdio.h. Un objet de type FILE * est appelé *flot de données* (en anglais, stream).

Déclaration d'un fichier **FILE *f1;**

On définit un pointeur f1 vers un fichier (à noter que « FILE » doit impérativement être en **majuscules**). Ce pointeur fournit l'adresse d'un enregistrement du fichier.

Les opérations disponibles en C pour **manipuler des fichiers** sont les suivantes :

1. création,
2. ouverture, fermeture,
3. lecture, écriture,
4. destruction,
5. renommage.

Ouvrir et fermer un fichier

5

► Ouverture d'un fichier

► `FILE * fopen ( const char * filename, const char * mode );`

► Fermeture d'un fichier

`int fclose ( FILE * stream );`

► Arguments

- `filename` : Chaîne de caractères contenant le nom du fichier
- `mode` : « modes » d'ouverture du fichier

► Modes

- « r » pour `lecture` (le fichier doit exister)
- « w » pour `écriture` (créer ou recréer un fichier vide)
- « a » pour `ajouter` à la fin d'un fichier (créer un fichier vide s'il n'existe pas)
- « b » considère le fichier comme « binaire » (pas comme une suite de caractère)
- « + » mode de mise à jour du fichier

Modes d'accès

"r"	ouverture d'un fichier texte en lecture
"w"	ouverture d'un fichier texte en écriture
"a"	ouverture d'un fichier texte en écriture à la fin
"rb"	ouverture d'un fichier binaire en lecture
"wb"	ouverture d'un fichier binaire en écriture
"ab"	ouverture d'un fichier binaire en écriture à la fin
"r+"	ouverture d'un fichier texte en lecture/écriture
"w+"	ouverture d'un fichier texte en lecture/écriture
"a+"	ouverture d'un fichier texte en lecture/écriture à la fin
"r+b"	ouverture d'un fichier binaire en lecture/écriture
"w+b"	ouverture d'un fichier binaire en lecture/écriture
"a+b"	ouverture d'un fichier binaire en lecture/écriture à la fin

Les fichiers

7

Exemples

```
FILE* pFile = fopen("monFichier.txt", "r") ; // ouverture en lecture
// instructions ....
fclose(pFile) ;
```

```
FILE* fic = fopen(nom_fic, "w"); // ouvrir en écriture
if(fic == NULL){
    printf("Impossible d'ouvrir le fichier %s\n", nom_fic);
    exit(1);
}
printf("..... Ouverture du fichier %s\n", nom_fic);
```

Retour

La fonction `fopen()` renvoie un pointeur vers un fichier (`FILE*`).

La fonction `fclose()` renvoie 0 si le fichier s'est correctement fermé.

EOF : si la fermeture a échoué. EOF est un *define*, nombre spécial, utilisé pour dire soit qu'il y a eu une erreur, soit que nous sommes arrivés à la fin du fichier.

Les fichiers

8

On distingue **deux techniques de gestion de fichiers** :

- **l'accès séquentiel**

consiste à traiter les informations séquentiellement, c'est-à-dire dans l'ordre où elles apparaissent (ou apparaîtront) dans le fichier ;

- **l'accès direct**

consiste à se placer immédiatement sur l'information souhaitée, sans avoir à parcourir celles qui la précèdent.

Ecrire dans un fichier

9

Fichier text

```
int fprintf ( FILE * fichier, const char * format, ... );
```

Fichier binaire

```
size_t fwrite (const void *ptr, size_t taille, size_t nbBloc, FILE  
*fichier);
```

Arguments

fichier: pointeur vers le fichier dans lequel écrire

format : chaîne de caractères à écrire dans le fichier.

Pour écrire le contenu d'une variable dans cette chaîne, vous devez utiliser des identifiants de formats tels que ceux-ci :

«**%d** » ou « **%i** » : écris un entier (int)

«**%f** » : écris un réel (float)

«**%c** » : écris un caractère

«**%s** » : écris une chaîne de caractère

Pour chaque identifiant inclus dans la chaîne, une variable de type correspondant doit être rajoutée dans la liste des arguments de la fonction.

Ecrire dans un fichier

- ▶ **int fputc(int c, FILE *stream);**
Ecrit `c` dans `stream`. Retourne `c`.
Fonction (macro) équivalente : `putc(c,stream)`
`fputc(c)` équivalent à `putc(c,stdout)`
- ▶ **int fputs(const char *s, FILE *stream);**
Ecrit `s` dans `stream`. Retourne les caractères écrits.
`fputs(s)` équivalent à `fputs(s,stdout)`
- ▶ **int putw(int n, FILE *fichier);**
 - ▶ Idem, `n` de type `int`, le pointeur avance du nombre de cases correspondant à la taille d'un entier (4 cases en C standard).
 - ▶ Retourne `n` si l'écriture s'est bien passée.

Ecrire dans un fichier

11

Exemple1

```
const char* fileName = "monfichier.txt" ;  
FILE* pFile = fopen(fileName, « w") ;  
int monEntier = 5;  
float monReel = 3.14;  
// \n est un caractère spéciale écrivant un saut de ligne dans le fichier  
fprintf(pFile, "Mon fichier contient %d ,%f \n", monEntier, monReel) ;  
fclose(pFile) ;
```

Retour

La fonction fprintf renvoie le nombre de caractères écrits dans le fichier.

Ecrire dans un fichier

12

Exemple 2

```
int main(){
    char fileName [20];
    int n;
    printf("Nom du fichier " );
    scanf(" %s", fileName );
    FILE* pFile = fopen(fileName, "w") ;
    do{
        printf(« Donner un entier" );
        scanf(" %d", &n );
        if(n!=0)
            fprintf(pFile, « %d \n», n);
    }while(n!=0);

    fclose(pFile);
}
```

Lire un fichier

13

```
int fscanf ( FILE * stream, const char * format, ... );
```

Arguments

stream : pointeur vers le fichier à lire

format : chaîne de caractères à lire dans le fichier

Pour lire le contenu d'une variable dans cette chaîne, vous devez utiliser des identifiants de formats tel que ceux-ci :

«%d » ou « %i » : lis un entier (int)

«%f » : lis un réel (float)

«%c » : lis un caractère

«%s » : lis une chaîne de caractère

Pour chaque identifiant inclus dans la chaîne, une variable de stockage de type correspondant doit être rajoutée dans la liste des arguments de la fonction.

Lire un fichier

- ▶ `int fgetc(FILE *stream);`
 - ▶ Retourne le caractère suivant de stream
 - ▶ EOF si une erreur intervient
 - ▶ Fonction équivalente : `int getc(FILE *stream);`
 - ▶ `int getchar()` “équivalent à” `getc(stdin)`
- ▶ `char *fgets(char *s, int n, FILE *stream);`
 - ▶ Lit au plus n-1 caractères de stream
 - ▶ S'arrête sur une fin de ligne.
 - ▶ Retourne s.

Lire un fichier

- ▶ `char *gets(char *s);`
 - ▶ Lit l'entrée standard jusqu'au prochain `'\0'`
 - ▶ Equivalent à `scanf("%s\n",s)`

```
char chaine[20] = "";
```

```
printf("Entrez une chaine:\n");  
fgets(chaine, sizeof(chaine), stdin);  
printf("Chaine: %s.\n", chaine);
```

Lire un fichier

16

Exemples

```
const char* fileName = "monfichier.txt" ;  
FILE* pFile = fopen(fileName, "rb") ;  
int monEntier ;  
float monReel;  
while(feof(pFile))    // int feof(FILE *stream) test la fin du fichier  
{  
    fscanf(pfile, "%d", &monEntier) ;  
    fscanf(pfile, "%f", &monReel) ;  
    // Traitement sur les données récupérées  
}  
fclose(pFile) ;
```

Retour

La fonction `fscanf()` renvoie le nombre de variables correctement remplies.

Lire un fichier

17

Exemple 2

```
int main(){
    char fileName [20];
    int n;
    printf("Nom du fichier à lire" );
    scanf(" %s", fileName );
    FILE* pFile = fopen(fileName, "r" ) ;
    do{
        fscanf( pFile , " %d", &n );
        if(!feof(pFile))
            printf(" %d \n ", n);
    }while (!feof(pFile));

    fclose(pFile);
}
```

```
#include <stdio.h> int
main () {
    FILE *fp;
    int c;
    fp = fopen("file.txt", "r");
    while(1) {
        c = fgetc(fp);
        if( !feof(fp) )
            printf("%c", c);
    }
    fclose(fp);
    return(0);
}
```

Renommer un fichier

```
int rename(char *oldname, char *newname) ;
```

Retourne 0 si le renommage s'est correctement passé.

Exemple

```
char oldname[] = "file.txt";  
char newname[] = "newfile.txt";  
int rep = rename(oldname, newname);  
if(rep== 0)  
    printf("Fichier renommé");  
else  
    printf("Erreur : fichier non renommé ");  
}
```

Accès direct: positionnement du pointeur

19

- ▶ Les différentes fonctions d'entrées-sorties permettent d'accéder à un fichier en mode *séquentiel* : les données du fichier sont lues ou écrites les unes à la suite des autres. Il est également possible d'accéder à un fichier en *mode direct*, c'est-à-dire que l'on peut se positionner à n'importe quel endroit du fichier.
- ▶ La fonction **fseek** permet de se positionner à un endroit précis dans le fichier;
int fseek(FILE *f, long déplacement, int origine);
- ▶ La variable *déplacement* détermine la nouvelle position dans le fichier. Il s'agit d'un déplacement relatif par rapport à l'*origine* ; il est compté en nombre d'octets. La variable *origine* peut prendre trois valeurs :
 - ▶ SEEK_SET (égale à 0) : début du fichier ;
 - ▶ SEEK_CUR (égale à 1) : position courante ;
 - ▶ SEEK_END (égale à 2) : fin du fichier.
- ▶ La fonction **int rewind(FILE *f);**
permet de se positionner au début du fichier. Elle est équivalente à `fseek(f, 0, SEEK_SET);`
- ▶ La fonction **long ftell(FILE *f);**
retourne la position courante dans le fichier (en nombre d'octets depuis l'origine).

Positionnement du pointeur au début du fichier

20

```
#include <stdio.h>

int main () {
    FILE *fp; fp = fopen("file.txt","w+");
    fputs(" C'est un en langage C", fp);
    fseek( fp, 8, SEEK_SET );
    fputs(" exemple ", fp);
    fseek(fp, -10 * sizeof(int), SEEK_END);
    printf("\n position %d", ftell(fp)); // ftell : renvoie la position
    courante
    rewind(fp); //retour au debut du fichier
    fclose(fp);
    return 0;
}
```

Ecriture dans fichier binaire: fwrite

21

size_t fwrite (const void *adrDebut, size_t taille, size_t nbBlocs, FILE *fichier);

- ▶ **adrDebut** : adresse de début de bloc à écrire. son type est sans importance.
- ▶ **Taille**: taille d'un bloc .
- ▶ **nbBlocs** : nombre de bloc à écrire.
- ▶ **fichier** : pointeur sur le flux (pointeur sur FILE dans notre cas).
- ▶ **Valeur retournée** La fonction fwrite retourne le nombre d'éléments qu'elle a réussi à écrire correctement dans le flux pointé par *fichier*.
- ▶ Ecrit à partir de la position courante du pointeur fichier nb_bloc X taille octets lus à partir de l'adresse fichir. Le pointeur fichier avance d'autant.
- ▶ Retourne le nombre de blocs écrits.

Exemple: taille_bloc=4 (taille d'un entier en C), nb_bloc=3, écriture de 3 entiers.

```
int tab[10] ;  
fwrite(tab, 4, 3, fichier) ;
```

```
int n=4;  
fwrite(&n, sizeof(int), 1, pFile); //écriture d'un entier
```

Ecriture dans fichier binaire:

22

fwrite

Exemple

```
#include <stdio.h>
int main(){
    char nomFich[20];
    int n;
    printf("Nom du fichier à lire \n" );
    scanf(" %s", fileName );
    FILE* pFile = fopen(nomFich, " w") ;
    do{
        printf(« Donner un entier" );
        scanf(" %d", &n );
        if(n!=0)
            fwrite(&n, sizeof(int), 1, pFile);
    }while(n!=0);

    fclose(pFile);
}
```

```
# define N 100
int main(){ int i;
float a[N], b[N];
FILE *fichier;
for (i=0; i<N; i++) a[i]=i;
fichier =fopen(« exe1.bin", "wb");
if(fichier !=NULL)
{fwrite(a, sizeof(float), N, fichier);
fclose(fichier);}
fichier= fopen(« exe2.bin", "rb");
if(fichier !=NULL)
{fread(b, sizeof(float), N, fichier);
fclose(fichier);}
}
```

Lecture d'un fichier binaire: fread

size_t **fread**(const void *adrDebut, size_t taille, size_t nbBlocs, FILE *fichier);

- **adrDebut** : adresse de début de la zone mémoire à remplir.
- **Taille**: taille d'un bloc .
- **nbBlocs** : nombre de bloc à lire.
- **fichier** : pointeur sur fichier
- **Valeur retournée** La fonction fread retourne le nombre d'éléments qu'elle a réussi à lire correctement dans le flux pointé par *fichier*.

```
Int n;  
fread(&n, sizeof(int), 1, pFile);
```

Lecture d'un fichier binaire: fread

24

Pour lire dans un fichier binaire, on lit en général dans le fichier les éléments d'un tableau. Chaque élément du tableau est appelé un *bloc*. *Chaque bloc possède une **taille en octets**. Un **char** correspond à 1 octet, un **float** correspond à 4 octets*, etc. On utilise de préférence la fonction `sizeof`

La fonction de lecture `fread` prend en paramètre le **tableau**, la **taille** de chaque bloc, le **nombre** de blocs à lire (nombre d'éléments du tableau), et le **pointeur** de fichiers. La taille physique du tableau doit être au moins égale au nombre de blocs lus, pour éviter une erreur mémoire.

La fonction `fread` transfère les données du fichier binaire vers le tableau.

On peut **lire une variable x avec la fonction fread**. Il suffit pour cela de **mettre** l'adresse `&x` de la variable à la place du tableau et de mettre le nombre de blocs égal à 1 ; les données sont alors transférées dans la variable.

La fonction `fread` retourne le **nombre d'éléments** effectivement lus.

Lecture d'un fichier binaire : 25

fread Exemple

```
#include <stdio.h>
int main(){
    char nomFich[20];
    int n;
    printf("Nom du fichier à lire \n" );
    scanf(" %s", fileName );
    FILE* pFile = fopen(nomFich, "r" ) ;
    do{
        fread(&n, sizeof(int), 1, pFile);
        if(!feof(pFile))
            printf(" %d \n ", n);
    }while (!feof(pFile));
    fclose(pFile);
}
```

Autres fonctions

26

`fgets(fichier, taille , chaine)`: lit une chaine de caractères dans un fichier

`FILE*f;`

`char ch[90];`

`fgets(ch, 90, f);`

`fputs(fichier, chaine)`: écrit une chaine de caractères dans un fichier

`FILE*f;`

`char *ch ; ch = « texte à écrire »;`

`fputs(ch, f);`

la fonction **`int feof(FILE *fichier)`**

retourne 0 tant que la fin du fichier n'est pas atteinte.

Lecture d'un fichier binaire : fread

27

Exemple 2

float Chargement(char* nomFichier, int *adrNbElem)*

```
{  
    int n, ret;  
    float *tableau;  
    FILE *fp;  
    if ((fp = fopen(nomFichier, "r")) == NULL)  
    {  
        printf("Erreur :");  
        puts("fichier introuvable ou droits insuffisants");  
        exit(1);  
    }  
    fread(&n sizeof(int), 1, fp);          /* on lit le nombre d'éléments */  
    *adrNbElem = n;                        /* passage par adresse */  
    /* le nombre d'éléments connu, on alloue le tableau : */  
    tableau = (float*)malloc(n*sizeof(float)) ; /* allocation */  
    ret = fread(tableau, sizeof(float), n, fp);  
    if (ret!=n)                            /* lecture des éléments */  
        puts("Erreur de lecture ou fin de fichier !");  
    fclose(fp);                            /* fermeture du fichier (obligatoire) */  
    return tableau;  
}
```

Ecriture sur un fichier binaire : fwrite

28

Exemple

```
#include <stdio.h>

#include <stdlib.h>

#define NB 50

#define F_SORTIE "sortie «

int main() {
    FILE *f_in, *f_out;
    int *tab; int i;
    tab = (int*)malloc(NB * sizeof(int));
    for (i = 0 ; i < NB; i++)    tab[i] = i;
                                /* ecriture du tableau dans F_SORTIE */
    if ((f_out = fopen(F_SORTIE, "w")) == NULL) {
        printf("\nImpossible d'ecrire dans le fichier %s\n",F_SORTIE);
    }
    fwrite(tab, NB * sizeof(int), 1, f_out);
    fclose(f_out);
}
```

Positionnement du pointeur au début du fichier

29

```
/* lecture dans F_SORTIE */
if ((f_in = fopen(F_SORTIE, "r")) == NULL) {
    printf("\nImpossible de lire dans le fichier %s\n", F_SORTIE);
}

/* on se positionne a la fin du fichier */
fseek(f_in, 0, SEEK_END); printf("\n position %ld", ftell(f_in));

/* déplacement de 10 int en arriere */
fseek(f_in, -10 * sizeof(int), SEEK_END);
printf("\n position %ld", ftell(f_in)); // ftell : renvoie la position courante
fread(&i, sizeof(i), 1, f_in); printf("\t i = %d", i);

/* retour au debut du fichier */
rewind(f_in); printf("\n position %ld", ftell(f_in));
fread(&i, sizeof(i), 1, f_in); printf("\t i = %d", i);

/* déplacement de 5 int en avant */
fseek(f_in, 5 * sizeof(int), SEEK_CUR); printf("\n position %ld", ftell(f_in));
fread(&i, sizeof(i), 1, f_in); printf("\t i = %d\n", i); fclose(f_in);
return(EXIT_SUCCESS);
}
```